

Universidad Gerardo Barrios
Facultad de Ciencia y Tecnología



Modulo:
Desarrollo de Aplicaciones Inteligentes

Nombre del proyecto:
Best.IA

Docente:
Marco Antonio Arévalo Zambrano

Presentado por:
PLACIDO ISMAEL LUNA ARGUETA SMIS071121
NASER AUDELI CLAROS RIVERA SMIS099121
SOFIA GISSELL HERNADEZ ASCENCIO SMIS015421

San Miguel, 15 de Agosto de 2026

1. Descripción del servicio

BEST.IA es una aplicación web de acompañamiento emocional para estudiantes universitarios. El estudiante escribe un mensaje en un chat privado y el sistema ejecuta un flujo compuesto por:

1. Validación del mensaje recibido.
2. Análisis emocional mediante un servicio local basado en pysentimiento.
3. Detección local de posibles señales de crisis.
4. Generación de una respuesta conversacional mediante Gemini/Gemma.
5. Persistencia del mensaje y la respuesta en Supabase.
6. Creación de una alerta para el psicólogo cuando se detecta una posible crisis.

Flujo crítico seleccionado

Se seleccionó el endpoint:

```
POST /api/chat
```

Este endpoint es crítico porque concentra la validación, el análisis emocional, la detección de crisis y la generación de la respuesta para el estudiante.

Ejemplo de entrada

```
{  
  "message": "Me siento muy estresado por los estudios"  
}
```

Ejemplo de respuesta exitosa

```
{  
  "content": "Respuesta empática generada por BEST.IA",  
  "emotion": "neg",  
  "emotionScore": 0.972,  
  "blocked": false,  
  "crisisDetected": false,  
  "model": "gemma-4-26b-a4b-it"
```

```
}
```

2. Preguntas de observabilidad

Las preguntas que se utilizarán para observar el sistema son:

- ¿Cuánto tarda el endpoint `/api/chat` en responder?
- ¿Qué parte del flujo consume más tiempo: validación, análisis emocional o modelo generativo?
- ¿Cuántas solicitudes terminan correctamente y cuántas con error?
- ¿Qué tipo de error ocurre cuando el servicio emocional no está disponible?
- ¿Qué versión del modelo se utilizó en cada solicitud?
- ¿Cuántas solicitudes activan una detección de crisis?
- ¿El sistema conserva un identificador para relacionar una solicitud con sus logs?
- ¿La respuesta se guarda correctamente en Supabase?
- ¿La alerta se crea cuando `crisisDetected` es verdadero?

3. Instrumentación propuesta

Para cada solicitud se debe registrar un evento estructurado. No se registra el mensaje completo porque puede contener información sensible.

Campos mínimos

Campo	Descripción
<code>request_id</code>	Identificador único de la solicitud
<code>route</code>	Ruta ejecutada, por ejemplo <code>/api/chat</code>
<code>status</code>	Resultado general: <code>success</code> o <code>error</code>

duration_ms	Duración total de la solicitud
model	Modelo generativo utilizado
emotion	Emoción detectada, sin guardar el texto original
crisis_detected	Indicador booleano de posible crisis
error_type	Tipo de error controlado, si existe
timestamp	Fecha y hora del evento

Ejemplo de log exitoso

```
{
  {"event":"api_request","request_id":"226c15b3-5e07-42d9-a348-
bca6d203061a","route":"/api/chat","status":"success","duration_ms":15966,"model":"gemma-4-
26b-a4b-it","emotion":"neg","crisis_detected":false,"error_type":null,"timestamp":"2026-08-
16T02:18:41.068Z"}
}
```

Ejemplo de error controlado

```
{"event":"api_request","request_id":"b964aa55-dad6-4c1f-8373-
e38a7a6b90d2","route":"/api/chat","status":"error","duration_ms":4,"model":null,"emotion":null,"cri
sis_detected":false,"error_type":"VALIDATION_ERROR","timestamp":"2026-08-
16T03:32:59.421Z"}
```

4. Privacidad y seguridad

BEST.IA procesa información relacionada con el bienestar emocional, por lo que los logs deben minimizar los datos almacenados.

Datos que no se registran

- Contraseñas.
- Tokens y claves de API.

- Encabezados de autorización.
- Cadenas de conexión de Supabase.
- El contenido completo del mensaje del estudiante.
- Correos electrónicos o nombres completos innecesarios.
- Respuestas completas del modelo cuando puedan contener información sensible.

Datos que sí se registran

- Identificador técnico de la solicitud.
- Ruta ejecutada.
- Duración.
- Estado de la operación.
- Tipo de error.
- Modelo utilizado.
- Emoción y bandera de crisis, como datos resumidos.

Esto permite investigar fallos sin exponer el contenido privado de las conversaciones.

5. Escenario de medición

Para establecer una línea base de rendimiento del endpoint `/api/chat`, se ejecutó un script de medición (`scripts/measure_chat.py`) que realizó 20 solicitudes secuenciales con un mensaje representativo de uso normal ("Me siento cansado por la carga académica"). El script midió el tiempo de respuesta de cada solicitud de extremo a extremo, capturando el código de estado HTTP, la emoción detectada y el indicador de crisis devueltos por la API.

Resultados obtenidos (n = 20):

Métrica	Valor
Solicitudes exitosas	20 / 20 (100%)
Tasa de error	0.0%
Mínimo	9,658.77 ms
Máximo	15,077.30 ms
Promedio	11,945.08 ms
Desviación estándar	1,549.56 ms
p50 (mediana)	12,080.62 ms
p95	13,875.82 ms

Las 20 solicitudes obtuvieron código de respuesta 200, con emotion: "neg" y crisis_detected: false de forma consistente, dado que el mensaje de prueba expresaba cansancio sin contener palabras clave de riesgo. No se registraron errores ni tiempos de espera agotados (timeouts) durante la medición.

La variabilidad observada (desviación estándar de ~1.5 segundos, con un rango entre 9.6 y 15.1 segundos) es consistente con la latencia esperada de una llamada de red a la API externa de Gemini, y no evidencia degradación progresiva ni picos anómalos: la diferencia entre el p50 y el p95 es de apenas 1.8 segundos, lo que indica un comportamiento estable bajo carga secuencial ligera

6. Línea base de rendimiento

Completar esta tabla con el resultado real del script:

Indicador	Resultado
Solicitudes realizadas	20
Solicitudes exitosas	20
Solicitudes con error	0
Tasa de error	0%
p50	"p50_ms": 12080.62,
p95	"p95_ms": 13875.82,
Tiempo máximo	"max_ms": 15077.3,

Interpretación

La línea base permitirá comparar futuras mejoras. Si el p95 es considerablemente mayor que el p50, significa que existe variabilidad en el tiempo de respuesta. Esto puede deberse a llamadas externas al servicio emocional, al modelo generativo o a la latencia de red.

7. Cuello de botella identificado

El análisis de instrumentación y la línea base de rendimiento permiten identificar con evidencia cuantitativa el principal cuello de botella del sistema: **la latencia del endpoint /api/chat está dominada casi en su totalidad por la llamada síncrona a la API de Gemini** para generar la respuesta conversacional.

Esto se evidencia al comparar los tres escenarios instrumentados:

Escenario	Duración	Camino de ejecución	Resultado
Mensaje normal (línea base, n = 20)	p50: 12,080.62 ms p95: 13,875.82 ms máx: 15,077.30 ms	Pasa por detección de emoción y llamada a Gemini	Respuesta generada, 0% de error
Mensaje de crisis (palabra clave detectada localmente)	35 ms	Se resuelve con detectCrisisLocally(), sin llamar a Gemini	Respuesta predefinida de emergencia
Body vacío ({})	4 ms	Falla en la validación de entrada, antes de cualquier lógica de negocio	Error 400, VALIDATION_ERROR

La diferencia es determinante: el camino que involucra a Gemini es entre 300 y 400 veces más lento que los caminos que se resuelven completamente en el servidor local (detección de crisis o validación de entrada fallida). Mientras que la validación y la detección de crisis responden en milisegundos de un solo dígito o de dos dígitos, cualquier mensaje que requiera una respuesta generativa completa tarda un mínimo de casi 10 segundos y hasta 15 segundos en el peor caso registrado.

Implicaciones:

- El tiempo de respuesta percibido por el usuario en el flujo conversacional normal (10-15 segundos) puede afectar la experiencia de uso, especialmente en un contexto de apoyo emocional donde la inmediatez de la respuesta importa.

- El sistema ya mitiga parcialmente este problema para el caso más crítico (mensajes de crisis), al resolverlos localmente sin depender de la latencia de Gemini, lo cual es una decisión de diseño correcta desde la perspectiva de seguridad y tiempo de respuesta.
- No se identificaron cuellos de botella adicionales atribuibles a la infraestructura propia (Docker, servicio de clasificación de emociones): la tasa de error de 0% y la baja dispersión de tiempos en la línea base sugieren que el sistema local no añade latencia significativa fuera de la llamada a la API externa.

Recomendación: cualquier esfuerzo de optimización de rendimiento debe enfocarse en el tiempo de respuesta de la llamada a Gemini (por ejemplo, mediante streaming de respuesta, caché de respuestas frecuentes, o un modelo más ligero para casos no críticos), ya que es el componente que concentra prácticamente toda la latencia del sistema.

8. Mejora aplicada o propuesta

Diagnóstico base: la latencia observada (10-15 segundos por solicitud) se concentra en la llamada síncrona a la API de Gemini, mientras que los caminos que se resuelven localmente (detección de crisis, validación de entrada, out-of-scope, prompt injection) responden en menos de 50 ms.

Mejora ya presente en el diseño actual (mitigación parcial): el sistema ya evita depender de Gemini para los casos más críticos de seguridad. La función `detectCrisisLocally()` intercepta el mensaje antes de llamar al modelo generativo cuando detecta palabras clave de riesgo (por ejemplo, "suicidio", "quiero morir", "hacerme daño") o cuando la emoción detectada es severa con `emotionScore >= 0.92`. Esto garantiza que la respuesta de emergencia se entregue en milisegundos, sin exponer al estudiante en riesgo a los ~12 segundos de latencia típicos del flujo normal. De igual forma, `isPromptInjection()` e `isOutOfScope()` filtran mensajes fuera de alcance antes de gastar tiempo y costo en la llamada al modelo.

Mejora propuesta (no implementada aún):

- Feedback progresivo en la interfaz: mostrar un indicador de "escribiendo..." o "pensando..." en `page-2.tsx` mientras se espera la respuesta de Gemini, ya que 10-15 segundos sin retroalimentación visual puede percibirse como que la aplicación se congeló.

- Streaming de respuesta: usar el modo de streaming de la API de Gemini (streamGenerateContent) en lugar de generateContent, para que el estudiante vea la respuesta construirse progresivamente en lugar de esperar el bloque completo. Esto no reduce la latencia total, pero mejora significativamente la latencia percibida.
- Timeout y reintento controlado: agregar un límite explícito de tiempo de espera (por ejemplo, 20 segundos) a la llamada fetch hacia Gemini, con un mensaje de error amigable si se excede, en lugar de dejar la solicitud abierta indefinidamente ante una falla de red.

Comparación antes/después: dado que estas mejoras no se han implementado en el código aún, no existe una medición "después" que comparar; se documentan como propuestas justificadas directamente por la evidencia de la línea base (sección 5) y el cuello de botella identificado (sección 8).

9. Plan de escalabilidad

Crecimiento considerado y restricción observada: el proyecto está diseñado para uso por estudiantes de una institución educativa. El crecimiento esperado es un aumento en el número de conversaciones simultáneas durante periodos de alta demanda (por ejemplo, semanas de evaluaciones). La restricción observada en la línea base es que cada solicitud a /api/chat ocupa entre 9.6 y 15 segundos de principal a fin, dominados por la latencia de la llamada externa a Gemini, lo cual limita el número de solicitudes que un mismo proceso puede atender de forma concurrente sin degradar el tiempo de respuesta percibido.

Primera mejora a realizar: antes de escalar infraestructura, la prioridad es reducir la latencia percibida (streaming de respuesta y feedback visual, sección 10) y agregar manejo de timeout, ya que actualmente una falla de red hacia Gemini podría dejar solicitudes colgadas sin límite de tiempo definido.

Usar caché, workers, colas o más instancias:

- **Caché:** conviene introducir caché de corto plazo si se detectan preguntas frecuentes o repetidas con la misma intención (por ejemplo, mensajes genéricos de saludo o consultas sobre el alcance del asistente), evitando llamadas innecesarias a Gemini para respuestas ya predecibles como las de isOutOfScope() e isPromptInjection(), que hoy ya se resuelven sin IA generativa.

- **Colas o workers:** serían necesarios si el volumen de solicitudes concurrentes empieza a saturar el proceso Node.js único que expone /api/chat, por ejemplo si varias decenas de estudiantes escriben simultáneamente durante un pico de uso.
- **Más instancias:** se justificaría cuando el tiempo de respuesta promedio empiece a degradarse por contención de recursos del servidor (CPU o memoria) más allá de la latencia inherente de Gemini, lo cual no se observó en esta línea base (baja desviación estándar de ~1.5 s con solicitudes secuenciales).

Impacto en memoria, datos, privacidad y costo:

- **Memoria:** el servicio actual no mantiene estado pesado en memoria por solicitud; el mayor riesgo de escalar sin control es acumular conexiones abiertas hacia Gemini y hacia el servicio de emociones si no se limitan.
- **Datos:** un mayor volumen de solicitudes implica más historiales de chat almacenados (ver chatSession, sendStudentMessage, saveBotMessage); esto debe planificarse con políticas de retención y anonimización, especialmente por tratarse de datos de bienestar emocional estudiantil.
- **Privacidad:** cualquier estrategia de caché debe evitar almacenar contenido textual sensible de los estudiantes; solo deben cachearse patrones o respuestas genéricas no ligadas a una persona.
- **Costo:** cada llamada a Gemini tiene un costo asociado por token; introducir caché para los casos ya filtrados localmente (out-of-scope, prompt injection, crisis) ya evita gasto innecesario, y cualquier mejora futura debe mantener ese principio de no llamar al modelo cuando no es necesario.

Indicador para decidir cuándo escalar: el indicador más directo es la tasa de error y el p95 de latencia del endpoint /api/chat medidos de forma continua. Un aumento sostenido del p95 muy por encima de los ~14 segundos observados en esta línea base, o una tasa de error superior al 1-2% (por timeouts o fallas de Gemini), señalaría la necesidad de intervenir con las estrategias de caché, colas o instancias adicionales descritas arriba.

10. Limitaciones y riesgos pendientes

- La medición de línea base se realizó con solicitudes secuenciales (una tras otra), no con concurrencia simultánea real; no se probó el comportamiento del sistema bajo múltiples estudiantes conectados al mismo tiempo.
- No se implementó aún streaming de respuesta, feedback visual de carga, ni manejo explícito de timeout en la llamada a Gemini; estas quedan como mejoras propuestas (sección 10).
- No se cuenta con monitoreo continuo en producción (por ejemplo, dashboards o alertas); la observabilidad actual se basa en logs puntuales y en el script de medición ejecutado manualmente.
- El plan de escalabilidad es una propuesta basada en la evidencia recolectada, pero no se implementó infraestructura adicional (caché, colas, múltiples instancias), conforme lo permite el alcance de esta evaluación.

11. Evidencias

- Captura del endpoint exitoso.

```
web-1 | - Network: http://172.18.0.3:3000
web-1 | ✓ Ready in 186ms
web-1 | ⚠ Invalid next.config.js options detected:
web-1 | ⚠   Unrecognized key(s) in object: 'swcMinify'
web-1 | ⚠ See more info here: https://nextjs.org/docs/messages/invalid-next-config
web-1 | {"event":"api_request","request_id":"aae565bd-b801-416c-8661-c9bf7c82c6e2","route":"/api/chat","status":"success","duration_ms":485,"model":null,"emotion":"neg","crisis_detected":true,"error_type":null,"timestamp":"2026-08-16T01:51:56.524Z"}
```

- Captura de un error controlado, body inválido.

```
web-1 | {"event":"api_request","request_id":"b964aa55-dad6-4c1f-8373-e38a7a6b90d2","route":"/api/chat","status":"error","duration_ms":4,"model":null,"emotion":null,"crisis_detected":false,"error_type":"VALIDATION_ERROR","timestamp":"2026-08-16T03:32:59.421Z"}
```

- Captura de la ejecución de 20 solicitudes.

```
PS C:\Users\placi\Desktop\PRE> python scripts/measure_chat.py | Tee-Object -FilePath docs/resultado-medicion.txt
Iniciando medicion: 20 solicitudes a http://localhost:3000/api/chat
[1/20] status=200 duration_ms=12889.6
[2/20] status=200 duration_ms=12212.39
[3/20] status=200 duration_ms=13598.81
[4/20] status=200 duration_ms=13070.52
[5/20] status=200 duration_ms=9782.25
[6/20] status=200 duration_ms=9737.3
[7/20] status=200 duration_ms=12886.2
[8/20] status=200 duration_ms=10895.49
[9/20] status=200 duration_ms=13224.36
[10/20] status=200 duration_ms=12076.44
[11/20] status=200 duration_ms=13875.82
[12/20] status=200 duration_ms=12084.8
[13/20] status=200 duration_ms=11386.81
[14/20] status=200 duration_ms=15077.3
[15/20] status=200 duration_ms=10667.76
[16/20] status=200 duration_ms=13272.61
[17/20] status=200 duration_ms=11412.23
[18/20] status=200 duration_ms=9658.77
[19/20] status=200 duration_ms=9923.91
[20/20] status=200 duration_ms=11168.29

=== RESULTADO FINAL ===
{
```

- Archivo docs/resultado-medicion.json.

```
resultado-medicion.txt U X TS chatSchema.test.ts JS jest.config.js ! ci.yml {} package-lock.json TS chatService.test.ts TS route
docs > resultado-medicion.txt
1  Iniciando medicion: 20 solicitudes a http://localhost:3000/api/chat
2  [1/20] status=200 duration_ms=12889.6
3  [2/20] status=200 duration_ms=12212.39
4  [3/20] status=200 duration_ms=13598.81
5  [4/20] status=200 duration_ms=13070.52
6  [5/20] status=200 duration_ms=9782.25
7  [6/20] status=200 duration_ms=9737.3
8  [7/20] status=200 duration_ms=12886.2
9  [8/20] status=200 duration_ms=10895.49
10 [9/20] status=200 duration_ms=13224.36
11 [10/20] status=200 duration_ms=12076.44
12 [11/20] status=200 duration_ms=13875.82
13 [12/20] status=200 duration_ms=12084.8
14 [13/20] status=200 duration_ms=11386.81
15 [14/20] status=200 duration_ms=15077.3
16 [15/20] status=200 duration_ms=10667.76
17 [16/20] status=200 duration_ms=13272.61
18 [17/20] status=200 duration_ms=11412.23
19 [18/20] status=200 duration_ms=9658.77
20 [19/20] status=200 duration_ms=9923.91
21 [20/20] status=200 duration_ms=11168.29
22
23 === RESULTADO FINAL ===
24 {
25   "endpoint": "http://localhost:3000/api/chat",
26   "requests": 20,
27   "successful_requests": 20,
28   "errors": 0,
29   "error_rate_percent": 0.0,
30   "p50_ms": 12080.62,
31   "p95_ms": 13875.82,
```

12. Checklist final

- ☐ El documento corresponde a BEST.IA.
- ☐ Se define POST /api/chat como flujo crítico.
- ☐ Se describen preguntas de observabilidad.
- ☐ Se documentan request_id, ruta, estado, duración, modelo y error.
- ☐ Se excluyen datos sensibles de los logs.
- ☐ Se incluye un evento exitoso.
- ☐ Se incluye un error controlado.
- ☐ Se ejecutan al menos 20 solicitudes.
- ☐ Se calculan p50, p95, máximo y tasa de error.
- ☐ Se identifica un cuello de botella.
- ☐ Se propone una mejora.
- ☐ Se incluye un plan de escalabilidad.
- ☐ Se agregan capturas legibles.
- ☐ Se agrega el enlace al repositorio.